

Laboratorio # 4

Base de Datos Tabulares

Introducción a Cassandra

Lenguaje CQL

Debido a las complejas estructuras que Cassandra tiene para ofrecer, es fundamental que exista una forma simple de manipular los datos almacenados. CQL (Cassandra Query Language) es una alternativa similar a SQL, que permite la creación y manipulación de datos en Cassandra.

Introducción a CQL



Probá los comandos de cada enunciado accediendo al cliente. Para ello debes ejecutar **cqlsh**

A través del comando `show host` es posible validar el estado de la conexión:



```
cqlsh> show host
Connected to Test Cluster at 127.0.0.1:9042.
```

Utiliza la función de autocompletar:



```
cqlsh> select * from system. (Presione tabulador)
```

Obtendrá las siguientes opciones:

IndexInfo	built_views	paxos	size_estimates
available_ranges	compaction_history	peer_events	sstable_activity
batches	hints	peers	views_builds_in_progress
batchlog	local	range_xfers	

Escriba cualquiera de las opciones, luego punto y coma (;) y presione <enter>. De esta manera, podrá consultar diversa información asociada al sistema.

Para efectos de la práctica, utilizaremos un servicio de música como ejemplo, que requiere de un Column Family canciones que compuesto por las columnas título, álbum, artista y una columna datos que contiene el archivo de audio real. La tabla utiliza el tipo UUID como clave primaria.

1. Cree un keyspace con el siguiente nombre: `demo_cql_music`



```
cqlsh> create KEYSPACE demo_cql_music WITH replication = {'class':  
'SimpleStrategy', 'replication_factor': '1'  
};  
cqlsh> USE demo_cql_music;
```

2. Cree el Column Family canciones:



```
cqlsh:demo_cql_music> create TABLE canciones (id uuid PRIMARY KEY,  
titulo text, album text, artista text, data blob);
```

En una base de datos relacional, se debería crear una tabla lista_reproduccion con una clave foránea a la tabla canciones. Sin embargo, en Cassandra se deben desnormalizar los datos porque los joins no son soportados.

3. Para representar los datos de una lista de reproducción, puede crear un Column Family como el siguiente:



```
cqlsh:demo_cql_music> create TABLE lista_reproduccion(id uuid,  
nro_cancion int, cancion_id uuid, titulo text, album text, artista  
text, PRIMARY KEY (id, nro_cancion));
```

La combinación del id y el nro_cancion identifican unívocamente a cada registro del Column Family lista_reproduccion. Se puede repetir el id de la lista para varios registros, sin embargo los nro_cancion deben ser distintos.

El UUID es útil para la secuenciación de los datos o incrementar automáticamente. Para el ejercicio, se utiliza el tipo int en el campo nro_cancion por simplicidad.

4. Ahora, veamos como insertar un registro en dicha tabla:



```
cqlsh:demo_cql_music> INSERT INTO lista_reproduccion (id, nro_cancion,  
cancion_id, titulo, artista, album) VALUES (62c36092-82a1-3a00-  
93d146196ee77204, 6, 7db1a490-5878-11e2-bcfd-0800200c9a66, 'Asilo',  
'Jorge Drexler', 'Salvavidas de Hielo');
```

Como ejercicio extra, inserte varios registros en el Column Family canciones utilizando la función now() para llenar el campo id. Luego obtenga los id generados e inserte nuevos registros en el Column Family lista_reproduccion.

5. Para hacer consultas, puede usar el siguiente comando:



```
cqlsh:demo_cql_music> SELECT * FROM lista_reproduccion;
```

6. Ejecute una consulta filtrada por artista:



```
cqlsh:demo_cql_music> SELECT * FROM lista_reproduccion WHERE  
artista='Jorge Drexler';
```

Cassandra rechaza la consulta porque el campo artista *no* es una columna de la clave de partición.

7. Al crear un índice para el campo artista, cassandra podrá responder la consulta:



```
cqlsh:demo_cql_music> CREATE INDEX indice_artista ON lista_reproduccion  
(artista);
```



```
cqlsh:demo_cql_music> SELECT * FROM lista_reproduccion WHERE  
artista='Jorge Drexler';
```

En el servicio de música se puede incluir la capacidad de etiquetar las canciones. Para etiquetar canciones utilice una colección set.

8. Debido a que en el Column Family lista_reproduccion ya existe debemos utilizar el comando ALTER TABLE:



```
cqlsh:demo_cql_music> ALTER TABLE lista_reproduccion ADD etiquetas set  
<text >;
```

9. Es posible actualizar registros ya existentes. Pruebe añadiendo etiquetas a las listas de reproducción:



```
cqlsh:demo_cql_music> UPDATE lista_reproduccion SET etiquetas =  
etiquetas + {'2017'} WHERE id = 62c36092-82a1-3a00-93d1-46196ee77204  
AND nro_cancion=6;  
cqlsh:demo_cql_music> UPDATE lista_reproduccion SET etiquetas =  
etiquetas + {'guitarra'} WHERE id = 62c36092-82a1-3a00-93d146196ee77204  
AND nro_cancion=6;  
  
cqlsh:demo_cql_music> UPDATE lista_reproduccion SET etiquetas =  
etiquetas + {'Mon Laferte'} WHERE id = 62c36092-82a1-3a00-  
93d146196ee77204 AND nro_cancion=6;
```

10. Al igual que como se hizo para el campo artista, para poder hacer búsquedas sobre la colección se debe indexar la misma:



```
cqlsh:demo_cql_music> CREATE INDEX indice_etiquetas ON
lista_reproduccion (etiquetas);
```

```
cqlsh:demo_cql_music> SELECT * FROM lista_reproduccion WHERE etiquetas
CONTAINS 'guitarra';
```

11. Es posible eliminar registros con el siguiente comando:



```
cqlsh:demo_cql_music> DELETE FROM lista_reproduccion WHERE id = 62c36092-82a1-3a00-93d1-46196ee77204 AND nro_cancion = 6;
```

12. Para eliminar el Column Family, utilice la siguiente sentencia:



```
cqlsh:demo_cql music> DROP TABLE lista_reproduccion ;
```

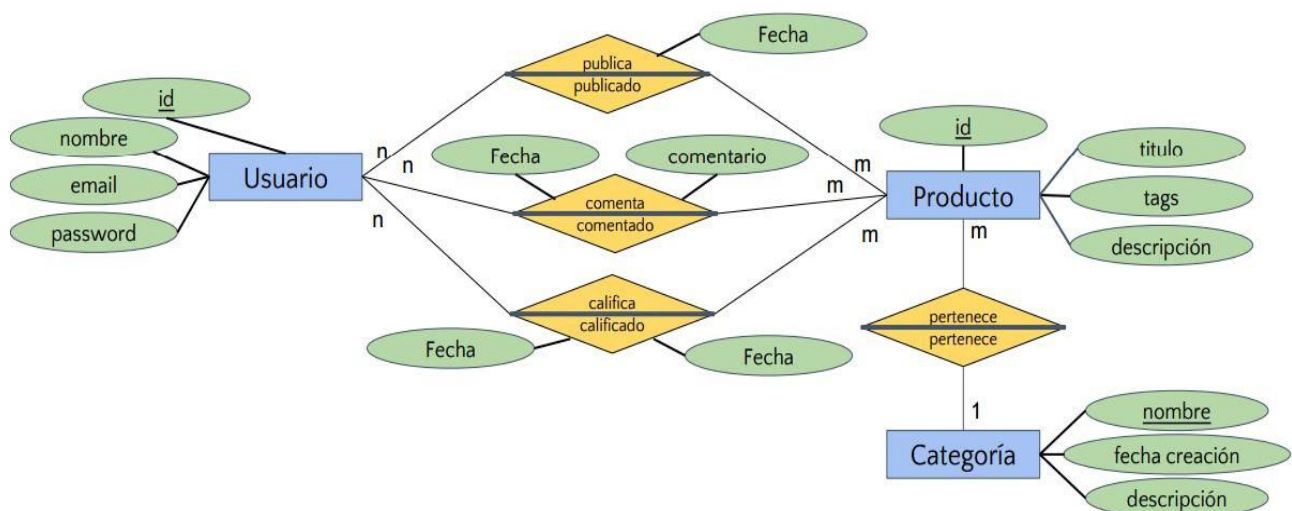
13. Para finalizar, elimine el Keyspace:



```
cqlsh:demo_cql_music> DROP KEYSPACE demo_cql_music;
```

PARTE PRÁCTICA

Usted es el arquitecto de un nuevo sitio de ecommerce que pretende revolucionar al mercado con sus rápidos tiempos de respuesta. Las entidades más importantes han sido definidas como se muestra a continuación:



Cree en Cassandra las estructuras necesarias para modelar el sistema descrito, tomando en cuenta que se debe dar respuesta a las siguientes consultas sobre los datos:

1. Obtener la información de usuario por ID de usuario
2. Obtener la información de producto por ID de producto
3. Obtener todos los productos publicados por un usuario particular (nombre usuario)
4. Obtener todos los usuarios que comentaron un producto particular (nombre producto)
5. Obtener que producto calificó un usuario 'X' por fecha

Tips:

- ✓ Recuerde que en cassandra se puede consultar solo por los campos que sean parte de la clave primaria o estén indexados
- ✓ Cassandra maneja colecciones : Set, List, Map
- ✓ Piense que datos necesita para responder cada consulta